Aerospace Science and Technology 168 (2026) 111085

Contents lists available at ScienceDirect

Aerospace Science and Technology

journal homepage: www.elsevier.com/locate/aescte

Adaptive model predictive control with extended state observer for unmanned aerial vehicle trajectory tracking

Kun Tian [a], Lin-Jie Hu [a], Chao Bai [b], Hai-Peng Ren [b,*]

[a] Electronics and Information Engineering, Xian Technological University, 710021, Xian, China
[b] National Key Laboratory of Land & Air Based Information Perception & Control, 710065, Xian, China

ARTICLE INFO

Communicated by Amit Kumar Sanyal

2025 MSC:
00–01
99-00

Keywords:
UAV trajectory tracking control
Neural network
Model predictive control
Extended state observer

ABSTRACT

Quadcopter Unmanned Aerial Vehicle (QUAV) trajectory tracking has a big challenge due to time-varying dynamics, and external disturbances such as wind. In this paper, an Adaptive Model Predictive Control with Extended State Observer (AMPCESO) framework is established. Due to the differences in the bandwidth of velocity and attitude loops, it is difficult to use a single one hidden layer network to identify two different time scale dynamics, therefore, this paper proposes dual single hidden layer neural networks in the predictive model in the model predictive control framework. To further strengthen robustness to the external disturbance and the internal uncertainty caused by the residual identification error of dual neural networks, an ESO is introduced and its outputs are incorporated into the dual neural network predictive models. The stability of the proposed AMPCESO is rigorously proved using the Lyapunov theorem. Numerical simulation and Hardware-in-the-Loop (HIL) experiment are conducted to validate the correctness of the theoretical analysis and the performance superiority of the proposed method.

1. Introduction

In recent years, the Quadcopter Unmanned Aerial Vehicle (QUAV) have attracted widespread attention, because of the wide applications, including cargo transportation [1], remote surveillance [2], emergency rescue [3], civil convenience [4], as well as in the battle field application. In the search, reconnaissance and surveillance applications, it is expected that the QUAV maintain a constant relative distance with the target, namely the standoff distance [5,6]. In this case, the QUAV reference trajectory is time variant, the environment is continuously changing, at the same time, the QUAV model inaccuracy makes the situation even worse. Therefore, QUAV trajectory tracking is a challenging task for both control theory and engineering control technique.

To address the challenges caused by external disturbance and inaccurate time-varying model, numerous researches have been carried out, such as robust Linear Quadratic Regulator control method [7], Particle Swarm Optimization - Proportion Integration Differentiation(PID)[8], Sliding Mode Control(SMC)[9], backstepping sliding mode control [10], nonlinear hierarchical control [11]. Fixed-time SMC has been proposed for flexible spacecraft [12] and rigid spacecraft [13] with uncertain inertia and external disturbances, however, their control gains are difficult to adjust to avoid controller singularity. Although these studies achieve significant progress in handling specific uncertainties, they remain limited in adaptively compensating for internal uncertainty and time-varying dynamics through real-time model updates. Consequently, it is challenging to simultaneously address time-varying dynamics and external disturbances without prior knowledge.

The predictive feature of Model Predictive Control (MPC) is a promising control approach for engineering applications, therefore, MPC has been applied extensively in UAV [14–27]. In [17], MPC for multi-input and multi-output systems were investigated for various scenarios, to achieve the complex trajectories tracking. To achieve smooth trajectory tracking, a MPC algorithm was proposed in [18] based on constrained linear errors and an enhanced model with an integrator to eliminate tracking errors caused by the external disturbance. In order to suppress the external disturbance, a robust time-varying constrained MPC was presented in [19] for controlling both the position and attitude of the QUAV. These approaches not only reduce the complexity of the controller by simplifying the optimization problem in the rolling optimization process, but also make the stability analysis be easy. However, the truncation of high-order terms in model linearization causes the degradation in trajectory tracking accuracy. In addition, the methods based on the linearized model might cause instability when the system operates at the point far from the pre-designed working point. To solve the model inaccuracy, the neural network model was proposed in MPC [20–26]. A MPC was presented in [26] using a pre-trained Feedforward Neural

* Corresponding author.
E-mail address: renhaipeng@xaut.edu.cn (H.-P. Ren).

https://doi.org/10.1016/j.ast.2025.111085
Received 25 February 2025; Received in revised form 4 September 2025; Accepted 12 October 2025
Available online 17 October 2025
1270-9638/© 2025 Elsevier Masson SAS. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Network (FNN) model to address model inaccuracy, which achieved better tracking performance as compared to that of Conventional MPC(CMPC) based on the linearized model. In [27], a method was proposed for time-varying nonlinear systems, utilizing dual functionally independent neural networks: a predictive network that replaces traditional prediction models and a control network that replaces the rolling optimization process to directly determine the optimal system input. To address the problem of the system with time-varying and disturbance, the disturbance observer (DO) method was proposed in [28], but it was difficult to tune the parameters. The interval observer (IO) demonstrated the robustness in [29], but it required prior knowledge of the disturbance. If the range of the disturbance cannot be known in advance, it is difficult to effectively observe the upper and lower bounds of the disturbance. Reference [30] established an adaptive nonsingular terminal sliding mode observer to estimate the external disturbances within the appointed settling time, however, the control performance is highly dependent on the considerable number of parameters selection. In [31], a parameter estimation-based observers (PEBO) was proposed to reformulating the state observation as a problem of parameter estimation using regressor equation. Based on generalized PEBO, a dynamic regressor extension and mixing (DREM) and least squares (LS), an adaptive observer is provided [32], but the convergence speed of the method is relatively slower than the Extended State Observer (ESO) method. The ESO has demonstrated nice performance to estimate the disturbance and provided the effective way to treat inaccuracy and time-varying model when combined with different types of controllers [33,34]. In [35], a Fast Finite Time Stable-Extended State Observer (FFTS-ESO) method was proposed to achieve high precision under the slow-varying or constant disturbances. In [36], an ESO was designed to estimate the inaccuracy and external disturbances of the system, and the estimated values were introduced into the cost function of Adaptive MPC (AMPC) in order to achieve improved performance. An CMPC with ESO was proposed in [37] to enhance the robustness to the external disturbance and system inaccuracy. Distinct from [37], the dual neural networks with pre-trained weight matrix and on-line adaption are proposed in this paper as the prediction model in MPC in order to obtain the nonlinear dynamic of the QUAV and suppress the mathematical model error caused by the time-varying factors, by this way, the performance is improved. To deal with the external disturbances like wind and internal uncertainty caused by the residual identification error of dual neural networks, the ESO is designed, and the outputs of ESO are fed into the dual neural networks to compensate the effect. The comparison results show the effectiveness and the superiority of the proposed method.

The contributions of this paper are as follows:

(1) The dual neural networks predictive model configure is proposed in this paper for the first time to identify the different time scale dynamics of velocity and attitude loops with only one hidden layer neural networks, and to achieve fast real time implementation on the commonly used control chip to avoid the complication using traditional single deep hidden layers neural network;
(2) ESO is integrated into the dual neural network predictive model in AMPCESO to compensate the external disturbance as well as the possible remaining model error, by this way, the problems caused by both inaccurate predictive model and the external disturbance are solved simultaneously;
(3) A new additive composite Lyapunov function is proposed for the first time to prove the stability of both the ESO and the AMPC in the proposed scheme.

The rest of this paper is organized as follows: The control problem and the mathematical model of QUAV considering wind disturbance are given in Section 2. the AMPCESO is given in detail in Section 3. The simulation and comparison results are given in Section 4 to show the effectiveness and the robustness of the proposed method. Conclusions are given in Section 5.

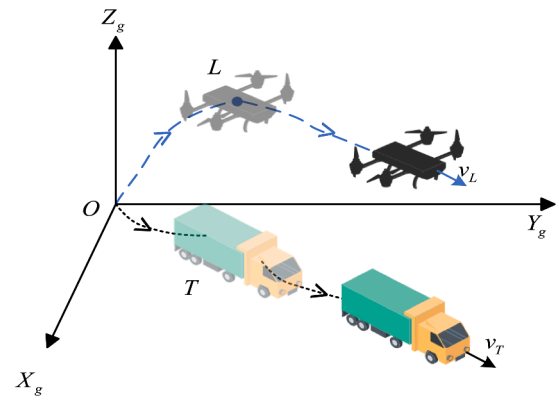


Fig. 1. The schematic diagram of the QUAV target tracking configuration.

2. Problem formulation

2.1. The trajectory tracking problem for QUAV

The positions of the QUAV and the tracked target are shown in Fig. 1. L represents the QUAV, and T represents the tracked target. υ_L, υ_T are the velocities of L and T, respectively. It is assumed that L is tracking directly above T when tracking the target. Therefore, the inertial frame $OX_gY_gZ_g$ is chosen to describe the position of the QUAV, with the origin point O; OX_g- axis is a certain direction in the horizontal plane, OY_g-axis is perpendicular to the OX_g- axis in the horizontal plane, and the OZ_g- axis is perpendicular to the plane X_gOY_g.

The task is to design a tracking controller such that L tracks directly above T under the disturbance and the inaccuracy.

2.2. QUAV model

According to the Newton Euler equation, The QUAV dynamic model with wind disturbance can be written as

$$\begin{cases} \dot{\Upsilon} = \upsilon \\ m\dot{\upsilon} = R_b^g F_t + mg\zeta + \mathbf{d}_F \\ \dot{\Theta} = B\sigma \\ J\dot{\sigma} = \Lambda - \sigma \times (J\sigma) + \mathbf{d}_M \end{cases} \tag{1}$$

where $\zeta = [0, 0, 1]^{\mathrm{T}}$. $\Upsilon = [x, y, z]^T$ and $\upsilon = [u, v, w]^T$ are the position and velocity vectors. m and $J = diag\{J_{xx}, J_{yy}, J_{zz}\}$ are the mass and inertia matrix of the QUAV, respectively. g is the gravity accelerant. $\Theta = [\phi, \theta, \varphi]^T$ is the attitude angle vector, including the roll angle ϕ, pitch angle θ and yaw angle φ. $\sigma = [p, q, r]^{\mathrm{T}}$ is the attitude angular rate vector. $F_t = \left[0, 0, T_t\right]^{\mathrm{T}}$ and $\Lambda = \left[\Lambda_x, \Lambda_y, \Lambda_z\right]^{\mathrm{T}}$ are the control force and torque acting on the QUAV respectively. R_b^g is the rotation matrix and B is the attitude kinematic matrix [29]. The forces and torque disturbances are denoted $\mathbf{d}_F = [F_X, F_Y, F_Z]^T$ and $\mathbf{d}_M = \left[M_{wp}, M_{wq}, M_{wr}\right]^T$, respectively, which are mainly observed by ESO. The detail of $\mathbf{d}_F$ and $\mathbf{d}_M$ are given by

$$\begin{cases} F_X = \frac{1}{2}\rho_w S_w {W_x}^2 \\ F_Y = \frac{1}{2}\rho_w S_w {W_y}^2 \\ F_Z = \frac{1}{2}\rho_w S_w {W_z}^2 \end{cases} \tag{2}$$

$$\begin{cases} M_{wp} = \frac{1}{2}\rho_w S_w d_q {W_x}^2 \\ M_{wq} = \frac{1}{2}\rho_w S_w d_q {W_y}^2 \\ M_{wr} = \frac{1}{2}\rho_w S_w d_q {W_z}^2 \end{cases} \tag{3}$$

where ρ_w is the air density, S_w is the area of the QUAV, and d_q is the length of QUAV. W_x, W_y, W_z are the wind velocity projections on body

axes, which are synthesized from three components:

$$\begin{cases} \mathrm{W}_x = \mathbf{v}_{avg}\cos(\varphi_{a,1})\cos(\varphi_{b,1}) + \mathbf{v}_j\cos(\varphi_{a,2})\cos(\varphi_{b,2}) + \mathbf{v}_m\cos(\varphi_{a,3})\cos(\varphi_{b,3}) \\ \mathrm{W}_y = \mathbf{v}_{avg}\sin(\varphi_{a,1})\cos(\varphi_{b,1}) + \mathbf{v}_j\sin(\varphi_{a,2})\cos(\varphi_{b,2}) + \mathbf{v}_m\sin(\varphi_{a,3})\cos(\varphi_{b,3}) \\ \mathrm{W}_z = \mathbf{v}_{avg}\sin(\varphi_{b,1}) + \mathbf{v}_j\sin(\varphi_{b,2}) + \mathbf{v}_m\sin(\varphi_{b,3}) \end{cases} \tag{4}$$

where $\mathbf{v}_{avg}$ is basic wind [38], $\mathbf{v}_j$ is gradual wind [39], $\mathbf{v}_m$ is stochastic wind [40], $\varphi_{a,i}(i = 1, 2, 3)$ is the direction angles of each wind component relative to the ground, and $\varphi_{b,i}(i = 1, 2, 3)$ is the angles between each wind component relative to the OX_g- axis.

The state variables are defined as $\mathbf{X}_L = [u, v, w, p, q, r]^T$. The control inputs are defined as $\mathbf{U}_\mathrm{o} = \left[T_t, \Lambda_x, \Lambda_y, \Lambda_z\right]^T$ and constrained by the following limits, $T_t = \left\{T_t \in R | T_t \in \left[0, T_{t\max}\right]\right\}$, $\Lambda_i = \left\{\Lambda_i \in R | \Lambda_i \in \left[\Lambda_{i\min}, \Lambda_{i\max}\right], i = x, y, z\right\}$.

By discretizing Eq. (1), we have

$$\begin{cases} \Upsilon(k+1) = \Upsilon(k) + v(k)\Delta T \\ v(k+1) = v(k) + \left(\frac{R_b^g(k)F_t(k)}{m} + g\zeta + \frac{\mathbf{d}_F(k)}{m}\right)\Delta T \\ \Theta(k+1) = \Theta(k) + (B(k)\sigma(k))\Delta T \\ \sigma(k+1) = \sigma(k) + \frac{(\Lambda(k)-\sigma(k)\times(J\sigma(k))+\mathbf{d}_M(k))}{J}\Delta T \end{cases} \tag{5}$$

where ΔT is the sampling time, which is determined primarily by the bandwidth of the UAV's attitude dynamics. Besides, to ensure consistency between simulations and hardware in practical hardware configuration (here it is the hardware in HIL), we aligned ΔT with the fixed sampling interval 4ms as used in our experimental platform.

Assumption 1. For continuous wind disturbance $\mathbf{v}_{wind}$ there exists positive scalar M, such that $\left\|\mathbf{v}_{wind}\right\| \le M$.

3. AMPCESO algorithm based on neural network

In this paper, a robust controller based on the framework of MPC is designed for the tracking problem. To deal with the complication of using traditional single deep hidden layers neural networks, a dual neural networks predictive model with only one hidden layer to identify the different time-scale dynamics of the velocity and attitude loops are used as the predictive model, which are online updated in order to achieve accurate tracking. To deal with the inaccuracy caused by predictive model and external disturbance, the ESO is designed to estimate and the estimated values are fed into the dual neural network predictive model and treated in the MPC framework.

3.1. Design of ESO

To address inaccuracy, the ESO is proposed to estimate the disturbance and improve the controller performance in this paper. The discrete system model (5) can be reorganized as:

$$\begin{cases} \mathbf{X}_L(k+1) = \mathbf{A}(k)\mathbf{X}_L(k) + \mathbf{B}(k)\mathbf{U}_\mathbf{o}(k) + \mathbf{B}_\mathbf{d}(k)\mathbf{d}(k) + \mathbf{D}(k) \\ \mathbf{y}(k) = \mathbf{C}(k)\mathbf{X}_L(k) \end{cases} \tag{6}$$

where $\mathbf{A}(k) = \mathbf{E}_{6\times6}$, $\mathbf{B}_d(k) = diag\left\{\frac{\Delta T}{m}\frac{\Delta T}{m}\frac{\Delta T}{m}\frac{\Delta T}{J_{xx}}\frac{\Delta T}{J_{yy}}\frac{\Delta T}{J_{zz}}\right\}$, $\mathbf{C}(k) = \mathbf{E}_{6\times6}$, $\mathbf{d}(k) = \left[\begin{array}{cc}\mathbf{d}_F(k) & \mathbf{d}_M(k)\end{array}\right]$, $\mathbf{E}$ is the identity matrix.

$$\mathbf{B}(k) = \Delta T\begin{bmatrix} \frac{-(\cos\varphi(k)\sin\theta(k)\cos\phi(k)+\sin\varphi(k)\sin\phi(k))}{m} & 0 & 0 & 0 \\ \frac{-(\sin\varphi(k)\sin\theta(k)\cos\phi(k)-\cos\varphi(k)\sin\phi(k))}{m} & 0 & 0 & 0 \\ \frac{-\cos\theta(k)\cos\phi(k)}{m} & 0 & 0 & 0 \\ 0 & \frac{1}{J_{xx}} & 0 & 0 \\ 0 & 0 & \frac{1}{J_{yy}} & 0 \\ 0 & 0 & 0 & \frac{1}{J_{zz}} \end{bmatrix},$$

$$\mathbf{D}(k) = \Delta T\left[00g\frac{-(J_{zz}-J_{yy})q(k)r(k)}{J_{xx}}\frac{-(J_{xx}-J_{zz})p(k)r(k)}{J_{yy}}\frac{-(J_{yy}-J_{xx})p(k)q(k)}{J_{zz}}\right]^T.$$

Denote $\mathbf{h}(k) = \mathbf{d}(k+1) - \mathbf{d}(k)$. Then, Eq. (6) can be rewritten as

$$\begin{cases} \mathbf{X}_L(k+1) = \mathbf{A}(k)\mathbf{X}_L(k) + \mathbf{B}(k)\mathbf{U}_\mathbf{o}(k) + \mathbf{B}_\mathbf{d}(k)\mathbf{d}(k) + \mathbf{D}(k) \\ \mathbf{d}(k+1) = \mathbf{h}(k) + \mathbf{d}(k) \\ \mathbf{y}(k) = \mathbf{C}(k)\mathbf{X}_L(k) \end{cases} \tag{7}$$

Defining $\xi_L(k) = \left[\mathbf{X}_L{}^\mathrm{T}(k), \mathbf{d}^\mathrm{T}(k)\right]^\mathrm{T}$, the extended discrete model can be obtained from Eq. (7) given by

$$\begin{cases} \xi_L(k+1) = \mathbf{A}_0(k)\xi_L(k) + \mathbf{B}_0(k)\mathbf{U}_\mathrm{o}(k) + \Delta_\mathbf{h}(k)\mathbf{h}(k) + \mathbf{D}_0(k) \\ \mathbf{y}(k) = \mathbf{C}_0(k)\xi_L(k) \end{cases} \tag{8}$$

where $\mathbf{A}_0(k) = \begin{bmatrix}\mathbf{A}(k)\mathbf{B}_d(k) \\ \mathbf{0}_{6\times6}\mathbf{E}_{6\times6}\end{bmatrix}$, $\mathbf{B}_0(k) = \begin{bmatrix}\mathbf{B}(k) \\ \mathbf{0}_{6\times4}\end{bmatrix}$, $\mathbf{\Delta}_\mathrm{h}(k) = \begin{bmatrix}\mathbf{0}_{6\times6} \\ \mathbf{E}_{6\times6}\end{bmatrix}$, $\mathbf{C}_0(k) = \left[\mathbf{C}(k)\mathbf{0}_{6\times6}\right]$, $\mathbf{D}_0(k) = \left[\mathbf{0}_{6\times1}\mathbf{D}(k)^T\right]^T$, $\mathbf{E}$ is the identity matrix.

$\mathbf{L}_1 = \begin{bmatrix} diag\{a, b, c, d, e, f\}_{6\times6} \\ ---------- \\ diag\{g, h, i, j, k, l\}_{6\times6} \end{bmatrix}_{12\times6}$ is the gain matrix of the ESO, the ESO for system (8) can be designed as

$$\begin{cases} \hat{\xi}_L(k+1) = \mathbf{A}_0(k)\hat{\xi}_L(k) + \mathbf{B}_0(k)\mathbf{U}_\mathrm{o}(k) + \Delta_\mathrm{h}(k)\mathbf{h}(k) + \mathbf{D}_0(k) + \mathbf{L}_1(\mathbf{y}(k) - \hat{\mathbf{y}}(k)) \\ \hat{\mathbf{y}}(k) = \mathbf{C}_0(k)\hat{\xi}_L(k) \end{cases} \tag{9}$$

where $\hat{\xi}_L(k) = \left[\hat{\mathbf{X}}_L(k)^\mathrm{T}, \hat{\mathbf{d}}(k)^\mathrm{T}\right]$ is the estimation of $\xi_L(k)$, $\hat{\mathbf{y}}(k)$ is the output estimation, $\hat{\mathbf{X}}_L(k) = \left[\hat{u}(k), \hat{v}(k), \hat{w}(k), \hat{p}(k), \hat{q}(k), \hat{r}(k)\right]^T$, $\hat{\mathbf{d}}(k) = \left[\hat{\mathbf{d}}_F^\mathrm{T}(k), \hat{\mathbf{d}}_M^\mathrm{T}(k)\right]^\mathrm{T} = [\hat{F}_X(k), \hat{F}_Y(k), \hat{F}_Z(k), \hat{M}_{wp}(k), \hat{M}_{wq}(k), \hat{M}_{wr}(k)]^\mathrm{T}$.

Theorem 1. *Considering system (5), if the gain matrix $\mathbf{L}_1$ of the designed ESO Eq. (9) satisfies $a > 2, b > 2, c > 2, d > 2, e > 2, f > 2$, $(a-1) < g \le \frac{a^2}{4}$, $(b-1) < h \le \frac{b^2}{4}$, $(c-1) < i \le \frac{c^2}{4}$, $(d-1) < j \le \frac{d^2}{4}$, $(e-1) < k \le \frac{e^2}{4}$, $(f-1) < l \le \frac{f^2}{4}$, such that $\left(\mathbf{A}_0 - \mathbf{L}_1\mathbf{C}_0\right)^T\left(\mathbf{A}_0 - \mathbf{L}_1\mathbf{C}_0\right) - \mathbf{E}$ is negative definite, then Eq. (9) will make the estimated variables approach the $\hat{\mathbf{d}}(k)$ in a finite time.*

The proof is shown in Appendix A.

3.2. MIMO neural network predictive model for UAV

The dual multiple-input multiple-output (MIMO) neural network predictive models are used to replace the traditional prediction model Eq. (5) in MPC to predict the system outputs. The structure of neural network prediction for velocity and attitude loops are shown in Fig. 2. The inputs of the neural network are defined as $\mathbf{P}_1(k) = [w(k) + \frac{\hat{F}_Z(k)}{m}\Delta T, \phi(k), \theta(k), T_t(k)]^T$ and $\mathbf{P}_2(k) = [p(k) + \frac{\hat{M}_{wp}(k)}{J_{xx}}\Delta T, q(k) + \frac{\hat{M}_{wq}(k)}{J_{yy}}\Delta T, r(k) + \frac{\hat{M}_{wr}(k)}{J_{zz}}\Delta T, \Lambda_x(k), \Lambda_y(k), \Lambda_z(k)]^T$. The outputs of the neural network are defined as $\mathbf{Y}_{o1}(k+1) = w_o(k+1)$ and $\mathbf{Y}_{o2}(k+1) = \left[p_o(k+1), q_o(k+1), r_o(k+1)\right]^T$, which represent the predicted states at the next time step.

The neural network models for velocity and attitude angular rate are represented as

$$\mathbf{Y}_{oi}(k+1) = \mathbf{LW}_i \times \tanh(\mathbf{IW}_i \times \mathbf{P}_i(k) + \mathbf{b}_{1i}) + \mathbf{b}_{2i}, i = 1, 2 \tag{10}$$

where $\mathbf{IW}_i$ and $\mathbf{LW}_i$ represent the weight matrixes of the input to hidden layer and the hidden to output layer, respectively. $\mathbf{b}_{1i}$ and $\mathbf{b}_{2i}$ represent the bias vectors of the input layer and the output layer, respectively. When $i = 1$, Eq. (10) represents the neural network for velocity, and when $i = 2$, it represents the neural network for attitude angular rate. The dimension of each vector/matrix are shown in Table 1.

3.3. Neural network pre-training

The neural network model without training cannot be used in the predictive model. Therefore, the neural network model is trained using the training data in an offline mode to obtain the pre-trained neural network for on-line predictive model. To obtain the training data, we select 8000 $\mathbf{U}_o$ randomly that satisfied the pre-defined constraints and

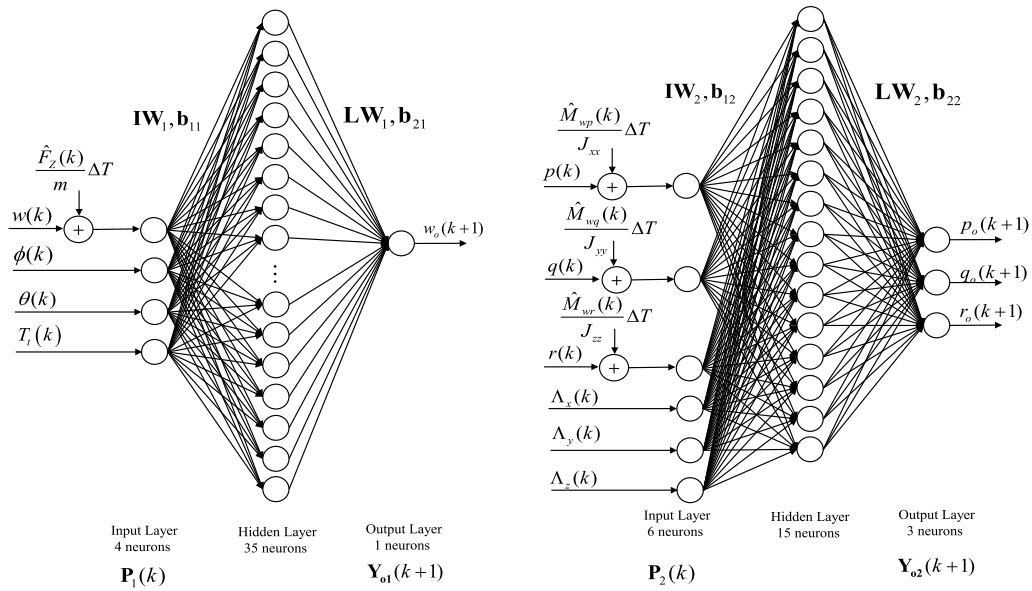


Fig. 2. The structure of the neural network prediction for velocity and attitude angular rate, (a) the neural network prediction for velocity; (b) the neural network prediction for attitude angular rate.

Table 1
Dimension of each vector/matrix in the neural network.

Vector/Matrix	$P_1(k)$	IW_1	LW_1	b_{11}	b_{21}	$Y_{o1}(k+1)$
Dimensionality (row × column)	4 × 1	35 × 4	1 × 35	35 × 1	1 × 1	1 × 1
Vector/Matrix	$\mathbf{P}_2(k)$	$\mathbf{IW}_2$	$\mathbf{LW}_2$	$\mathbf{b}_{12}$	$\mathbf{b}_{22}$	$\mathbf{Y}_{o2}(k+1)$
Dimensionality (row × column)	6 × 1	15 × 6	3 × 15	15 × 1	3 × 1	3 × 1

substituted them into Eq. (1), then record the input and output data set. 80 % of the set is used as training data and the rest 20 % is used as the test data. The data is normalized to avoid the risk of gradient disappearance or gradient explosion. Define the error function as

$$\mathbf{E}1_i = \frac{1}{2}\sum_{\mathrm{kn}=1}^{NN}\left[\mathbf{S}_i(kn) - \mathbf{Y}_{oi}(kn)\right]^2, i = 1, 2 \tag{11}$$

where NN is the number of the sampled training groups. $\mathbf{S}_1(kn)$ represents the expected velocity output at kn-th sampling time, $\mathbf{S}_2(kn)$ represents the expected attitude angular rate output at kn-th sampling time.

Further, $\mathbf{IW}_i(0), \mathbf{LW}_i(0), \mathbf{b}_{1i}(0), \mathbf{b}_{2i}(0)$ can be updated with the back-propagation algorithm, as follows

$$\begin{cases} \mathbf{IW}_i^{n+1}(0) = \mathbf{IW}_i^{n}(0) - lr_{\mathbf{IW}}\dfrac{\partial \mathbf{E}1_i^{n}}{\partial \mathbf{IW}_i^{n}(0)} \\ \mathbf{LW}_i^{n+1}(0) = \mathbf{LW}_i^{n}(0) - lr_{\mathbf{LW}}\dfrac{\partial \mathbf{E}1_i^{n}}{\partial \mathbf{LW}_i^{n}(0)} \\ \mathbf{b}_{1i}^{n+1}(0) = \mathbf{b}_{1i}^{n}(0) - lr_{\mathbf{b1}}\dfrac{\partial \mathbf{E}1_i^{n}}{\partial \mathbf{b}_{1i}^{n}(0)} \\ \mathbf{b}_{2i}^{n+1}(0) = \mathbf{b}_{2i}^{n}(0) - lr_{\mathbf{b2}}\dfrac{\partial \mathbf{E}1_i^{n}}{\partial \mathbf{b}_{2i}^{n}(0)} \end{cases}, n = 0, 1, 2, 3 \cdots N_{upper}; i = 1, 2 \tag{12}$$

where N_{upper} is the upper limit of training times; $lr_{\mathbf{IW}}$, $lr_{\mathbf{LW}}$, $lr_{\mathbf{b1}}$, $lr_{\mathbf{b2}}$, represent the learning rate of each coefficient, all of which are 0.001. $\mathbf{IW}_i^{n}(0), \mathbf{LW}_i^{n}(0), \mathbf{b}_{1i}^{n}(0), \mathbf{b}_{2i}^{n}(0)$ are the weight matrixes and bias vectors of the n-th iteration, $\mathbf{E}1_i^{n}$ is the error function of the n-th iteration.

According to Eq. (12), if the error function (11) is less than the desired threshold or the number of training times exceeds the given limit N_{upper}, the iteration is stopped. Subsequently, the 20 % test data is used for verification. If the error function is also less than the threshold, the neural network model approximates the nonlinear dynamical model the accepted accuracy. The final weight matrixes are used as $\mathbf{IW}_i(0), \mathbf{LW}_i(0), \mathbf{b}_{1i}(0), \mathbf{b}_{2i}(0)$.

3.4. MPC controller

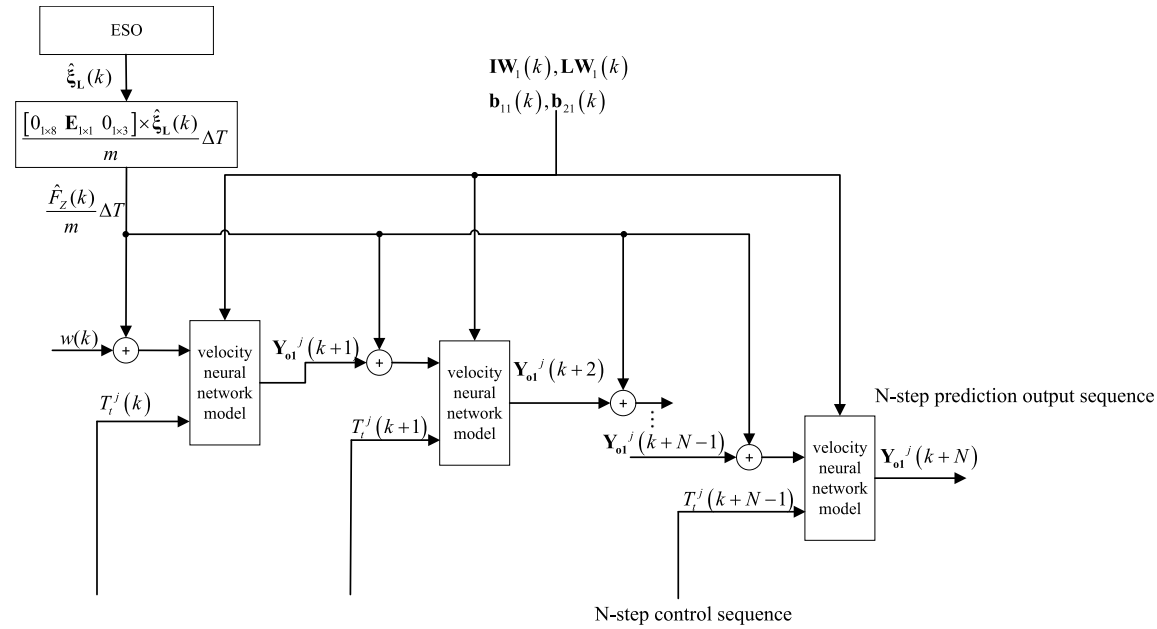
The essence of MPC is to predict the system output in the future, and to solve the optimal control at each step to minimize the cost function. In this paper, the neural network models are used to predict the system outputs over several N-steps in advance. The structure diagram of N-steps prediction using dual neural networks are shown in Fig. 3.

In Fig. 3, both neural networks employ the same ESO structure, with input data being $\frac{\hat{F}_Z(k)}{m}\Delta T$ and $\Delta T\begin{bmatrix} \frac{\hat{M}_{wp}(k)}{J_{xx}} & \frac{\hat{M}_{wq}(k)}{J_{yy}} & \frac{\hat{M}_{wr}(k)}{J_{zz}} \end{bmatrix}^T$, and the output data being $\mathbf{Y_{o1}}^j(k+1)$ and $\mathbf{Y_{o2}}^j(k+1)$ respectively. ESO estimates the disturbance vector $\hat{\xi}_L(k)$ and then subjected to a dimension transformation to obtain $\frac{\hat{F}_Z(k)}{m}\Delta T$ and $\Delta T\begin{bmatrix} \frac{\hat{M}_{wp}(k)}{J_{xx}} & \frac{\hat{M}_{wq}(k)}{J_{yy}} & \frac{\hat{M}_{wr}(k)}{J_{zz}} \end{bmatrix}^T$, $\frac{\hat{F}_Z(k)}{m}\Delta T$ and $\Delta T\begin{bmatrix} \frac{\hat{M}_{wp}(k)}{J_{xx}} & \frac{\hat{M}_{wq}(k)}{J_{yy}} & \frac{\hat{M}_{wr}(k)}{J_{zz}} \end{bmatrix}^T$ are superposed to the state vectors of the input of the neural network to predict $\mathbf{Y_{o1}}^j(k+1)$ and $\mathbf{Y_{o2}}^j(k+1)$, where j is the recurrent operator in the rolling optimization. Then, $\mathbf{Y_{o1}}^j(k+1)$, $\mathbf{Y_{o2}}^j(k+1)$, $\frac{\hat{F}_Z(k)}{m}\Delta T, \Delta T\begin{bmatrix} \frac{\hat{M}_{wp}(k)}{J_{xx}} & \frac{\hat{M}_{wq}(k)}{J_{yy}} & \frac{\hat{M}_{wr}(k)}{J_{zz}} \end{bmatrix}^T$, $T_t^j(k+1)$ and $\Lambda_i^j(k+1), i = x, y, z$ are used to predict $\mathbf{Y_{o1}}^j(k+1)$ and $\mathbf{Y_{o2}}^j(k+1)$), respectively. By this way, N-step prediction is achieved by repeating the above process N times.

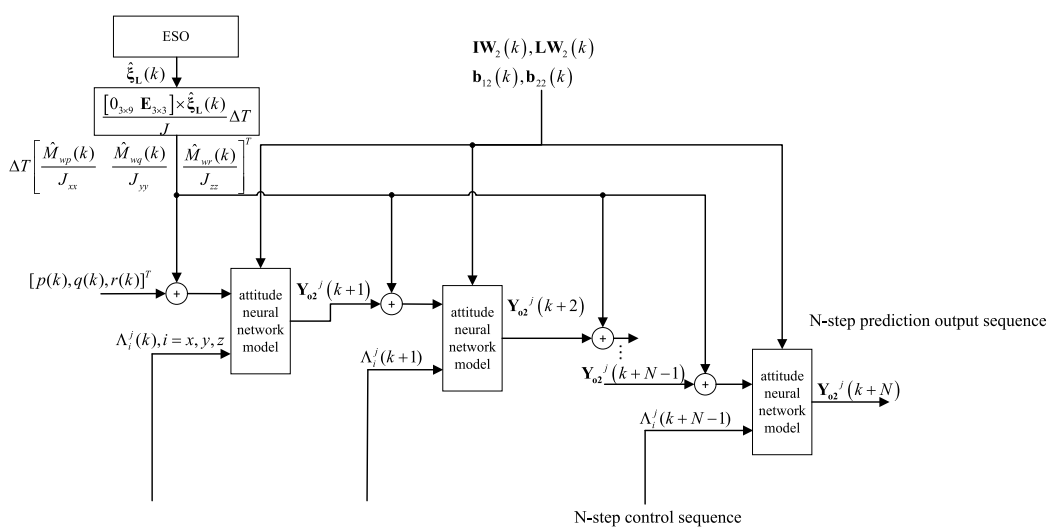
The above-mentioned control input $T_t\left(or\Lambda_i, i = x, y, z\right)$ can be obtained by the rolling optimization. Defining the cost function $\mathbf{J}_p(k)$ at the step k as

$$\mathbf{J}_p(k) = \sum_{i=1}^{N}\left[\mathbf{Y}_{\mathrm{o}p}(k+i|k) + \hat{\mathbf{d}}_p(k+i-1|k) - \mathbf{p}_p(k+i|k)\right]^2, p = 1, 2$$

$$s.t.\begin{cases} \mathbf{Y}_{\mathrm{o}p_{\min}} \le \mathbf{Y}_{\mathrm{o}p}(k+i) \le \mathbf{Y}_{\mathrm{o}p_{\max}}, i \in 1, \dots N \\ \hat{\mathbf{d}}_p = \hat{\mathbf{d}}_{\mathrm{F}}, \mathbf{p}_p = \mathbf{p}_T, p = 1 \\ \hat{\mathbf{d}}_p = \hat{\mathbf{d}}_{\mathrm{M}}, \mathbf{p}_p = \mathbf{p}_M, p = 2 \\ \hat{\mathbf{d}}_p(k+i-1|k) = \hat{d}_F(k), p = 1, i \in 1, \cdots N-1 \\ \hat{\mathbf{d}}_p(k+i-1|k) = \hat{d}_M(k), p = 2, i \in 1, \cdots N-1 \\ \hat{\mathbf{d}}_p(k+N-1|k) = 0 \end{cases} \tag{13}$$



(a)



(b)

Fig. 3. The structure diagram of N-steps prediction using neural networks,(a) Multi steps prediction of velocity; (b) Multi steps prediction of attitude angular rate.

where $(k+i|k)$ represents predicting the value of the $k+i$ step based on the information of the k step, $\hat{\mathbf{d}}_p$ represents the disturbance estimated by the ESO, $\mathbf{p}_p$ represents expected state, $\mathbf{p}_T$ and $\mathbf{p}_M$ represent expected velocity and expected angular rate, respectively.

According to the gradient descent algorithm, the control variable is updated by

$$\mathbf{U}_{op}{}^{j+1}(k+i-1|k) = \mathbf{U}_{op}{}^{j}(k+i-1|k) - lr_{uc}\frac{\partial \mathbf{J}_p{}^{j}(k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)}, j = 0,1,2\cdots J_{upper}, p=1,2 \tag{14}$$

where lr_{uc} is the step length of updating. $\mathbf{J}_p{}^{j}(k)$ is the cost function of the j-th iteration. J_{upper} is the upper limit of the number of iterations. $\mathbf{U}_{\mathrm{op}}{}^{j}$ is the control input sequence for the j-th iteration. When $i=1$, $\mathbf{U}_{\mathrm{op}}{}^{j} = T_t{}^{j}$ and when $i=2$, $\mathbf{U}_{\mathrm{op}}{}^{j} = \left[\Lambda_x^j, \Lambda_y^j, \Lambda_z^j\right]^T$.

Note that $\sum_{s=1}^{i-1}\frac{\partial \mathbf{Y}_{\mathrm{op}}{}^{j}(k+s|k)}{\partial \mathbf{U}_{\mathrm{op}}{}^{j}(k+i-1|k)}(i>1, p=1,2)$ is zero, thus the partial derivative of Eq. (13) with respect to the input control can be expressed as

$$\frac{\partial \mathbf{J}_p{}^{j}(k)}{\partial \mathbf{U}_{\mathrm{op}}{}^{j}(k+i-1|k)} = \sum_{s=i}^{N}\frac{\partial \mathbf{J}_p{}^{j}(k)}{\partial \mathbf{Y}_{\mathrm{op}}{}^{j}(k+s|k)}\frac{\partial \mathbf{Y}_{\mathrm{op}}{}^{j}(k+s|k)}{\partial \mathbf{U}_{\mathrm{op}}{}^{j}(k+i-1|k)}, p=1,2 \tag{15}$$

where

$$\frac{\partial \mathbf{J}_p{}^{j}(k)}{\partial \mathbf{Y}_{\mathrm{op}}{}^{j}(k+s|k)} = 2\left[\mathbf{Y}_{\mathrm{op}}{}^{j}(k+s|k) + \hat{\mathbf{d}}_p(k+s|k) - \mathbf{p}_p(k+s|k)\right], p=1,2 \tag{16}$$

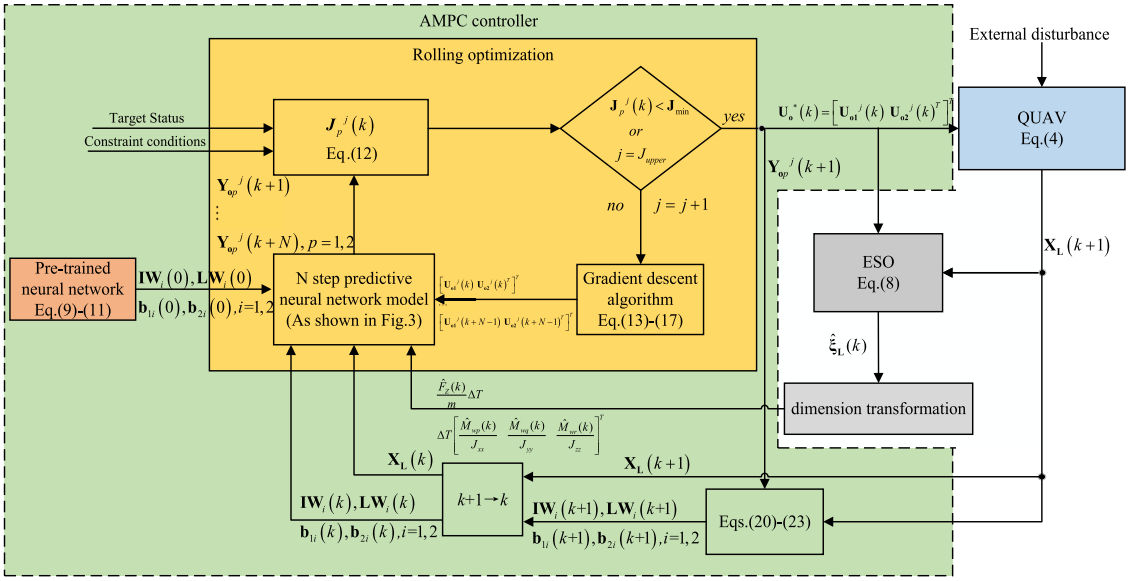


Fig. 4. The block diagram of the AMPCESO based on the adaptive neural network.

If $s = i$, we have

$$\begin{aligned}\frac{\partial \mathbf{Y}_{op}{}^{j}(k+s|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)} &= \mathbf{LW}_p(k)\times\mathbf{H}_p(k+i-1)\times\mathbf{IW}_{p(:,4)}(k), p=1\\ \frac{\partial \mathbf{Y}_{op}{}^{j}(k+s|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)} &= \mathbf{LW}_p(k)\times\mathbf{H}_p(k+i-1)\times\mathbf{IW}_{p(:,4:6)}(k), p=2\end{aligned} \tag{17}$$

where $\mathbf{H}_p(k+i-1) = \left(1-\tanh^2\left(\mathbf{IW}_p(k)\times\mathbf{P}_p(k+i-1)+\mathbf{b}_{1p}(k)\right)\right)$.

Otherwise

$$\begin{aligned}\frac{\partial \mathbf{Y}_{op}{}^{j}(k+s|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)} &= \mathbf{LW}_p(k)\times\mathbf{H}_p(k+i-1)\times\mathbf{IW}_{p(:,1)}(k)\frac{\partial \mathbf{Y}_{op}{}^{j}(k+s-1|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)}, p=1\\ \frac{\partial \mathbf{Y}_{op}{}^{j}(k+s|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)} &= \mathbf{LW}_p(k)\times\mathbf{H}_p(k+i-1)\times\mathbf{IW}_{p(:,1:3)}(k)\frac{\partial \mathbf{Y}_{op}{}^{j}(k+s-1|k)}{\partial \mathbf{U}_{op}{}^{j}(k+i-1|k)}, p=2\end{aligned} \tag{18}$$

It is noted that the optimization process stops when the number of iterations exceeds the given iteration limit or $\mathbf{J}_p{}^j(k)$ is less than the desired threshold, then the control variable for the k instance is derived as $\mathbf{U}_o{}^*(k) = \begin{bmatrix}\mathbf{U}_{o1}{}^j(k) & \mathbf{U}_{o2}{}^j(k)^T\end{bmatrix}^T$.

Theorem 2. *For system (5) satisfying Assumption 1, the AMPCESO scheme given in (14) ensure the tracking error converges asymptotically to zero.*

Proof. See the Appendix B. □

When the $(k+1)$-th practical output of system, i.e., $\mathbf{X}_L(k+1)$, is observed, the weight matrixes and bias vectors of the neural network can be updated online to make prediction be more accurate. For online updating of weights in the neural network, the cost function can be defined as

$$\mathbf{E2}_i = \frac{1}{2}\left[\mathbf{X}_{Li}(k+1)-\mathbf{Y}_{oi}{}^j(k+1)\right]^2, i=1,2. \tag{19}$$

where $\mathbf{X}_{L1}(k+1) = w(k+1)$, $\mathbf{X}_{L2}(k+1) = [p(k+1), q(k+1), r(k+1)]^T$.

According to the back propagation algorithm, the weight matrix updating formula of the hidden to output layer is

$$\begin{cases}\mathbf{LW}_i(k+1) = \mathbf{LW}_i(k)+\mathbf{\Delta LW}_i(k)\\ \mathbf{\Delta LW}_i(k) = -lr_{LW}\dfrac{\partial \mathbf{E2}_i(k)}{\partial \mathbf{LW}_i(k)} = -lr_{LW}\dfrac{\partial \mathbf{E2}_i(k)}{\partial \mathbf{Y}_{oi}{}^j(k+1)}\dfrac{\partial \mathbf{Y}_{oi}{}^j(k+1)}{\partial \mathbf{LW}_i(k)}\end{cases}, i=1,2. \tag{20}$$

where $\frac{\partial \mathbf{E2}_i(k)}{\partial \mathbf{Y}_{oi}{}^j(k+1)} = \mathbf{Y}_{oi}{}^j(k+1)-\mathbf{X}_{Li}(k+1)$, $\frac{\partial \mathbf{Y}_{oi}{}^j(k+1)}{\partial \mathbf{LW}_i(k)} = [\tanh(\mathbf{IW}_i(k)\times\mathbf{P}_i(k)+\mathbf{b}_{1i}(k))]^T$.

Then,

$$\begin{aligned}\mathbf{LW}_i(k+1) = \mathbf{LW}_i(k) - lr_{LW}&\left[\mathbf{Y}_{oi}{}^j(k+1)-\mathbf{X}_{Li}(k+1)\right]\\ &\times\left[\tanh\left(\mathbf{IW}_i(k)\times\mathbf{P}_i(k)+\mathbf{b}_{1i}(k)\right)\right]^T, i=1,2.\end{aligned} \tag{21}$$

Similarly, the bias vector updating formula of the hidden layer is

$$\mathbf{b}_{2i}(k+1) = \mathbf{b}_{2i}(k) - lr_{b_2}\left[\mathbf{Y}_{oi}{}^j(k+1)-\mathbf{X}_{Li}(k+1)\right], i=1,2. \tag{22}$$

The weight matrix updating formula of the input to hidden layer is

$$\begin{aligned}\mathbf{IW}_i(k+1) = \mathbf{IW}_i(k) - lr_{IW}&\mathbf{LW}_i{}^T(k)\times\left[\mathbf{Y}_{oi}{}^j(k+1)-\mathbf{X}_{Li}(k+1)\right]\\ &\cdot\left[1-\tanh^2(\mathbf{IW}_i(k)\times\mathbf{P}_i(k)+\mathbf{b}_{1i}(k))\right]\times\mathbf{P}_i{}^T(k), i=1,2.\end{aligned} \tag{23}$$

The bias vector updating formula of the input layer is

$$\begin{aligned}\mathbf{b}_{1i}(k+1) = \mathbf{b}_{1i}(k) - lr_{b_1}&\mathbf{LW}_i{}^T(k)\times\left[\mathbf{Y}_{oi}{}^j(k+1)-\mathbf{X}_{Li}(k+1)\right]\\ &\cdot\left[1-\tanh^2\left(\mathbf{IW}_i(k)\times\mathbf{P}_i(k)+\mathbf{b}_{1i}(k)\right)\right], i=1,2.\end{aligned} \tag{24}$$

To summarize, the block diagram of the AMPCESO control algorithm is shown in Fig. 4.

The ESO estimates the disturbance vector $\hat{\xi}_L(k)$ and then performs dimension transformation to obtain $\frac{\hat{F}_Z(k)}{m}\Delta T$ and $\Delta T\begin{bmatrix}\frac{\hat{M}_{wp}(k)}{J_{xx}} & \frac{\hat{M}_{wq}(k)}{J_{yy}} & \frac{\hat{M}_{wr}(k)}{J_{zz}}\end{bmatrix}^T$, then fed back to the neural network model for the output prediction and weights updating. The N-step prediction output $\mathbf{Y}_{op}{}^j(k+1),\cdots,\mathbf{Y}_{op}{}^j(k+N), p=1,2$ are predicted by the current neural network model according to the current input $\begin{bmatrix}\mathbf{U}_{o1}{}^j(k) & \mathbf{U}_{o2}{}^j(k)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}},\cdots,\begin{bmatrix}\mathbf{U}_{o1}{}^j(k+N-1) & \mathbf{U}_{o2}{}^j(k+N-1)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}}$ in each iteration. The $\mathbf{Y}_{op}{}^j(k+1),\cdots,\mathbf{Y}_{op}{}^j(k+N), p=1,2$ are used to calculate the cost function $\mathbf{J}_{\mathrm{p}}{}^j(k)$. According to the derivative of $\mathbf{J}_{\mathrm{p}}{}^j(k)$, the controller output $\begin{bmatrix}\mathbf{U}_{o1}{}^j(k) & \mathbf{U}_{o2}{}^j(k)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}}$, …, $\begin{bmatrix}\mathbf{U}_{o1}{}^j(k+N-1) & \mathbf{U}_{o2}{}^j(k+N-1)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}}$ is corrected by the gradient descent method. Repeat the above steps of predicting $\mathbf{Y}_{op}{}^j(k+1),\cdots,\mathbf{Y}_{op}{}^j(k+N), p=1,2$, calculating $\mathbf{J}_{\mathrm{p}}{}^j(k)$ and updating $\begin{bmatrix}\mathbf{U}_{o1}{}^j(k) & \mathbf{U}_{o2}{}^j(k)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}},\cdots,\begin{bmatrix}\mathbf{U}_{o1}{}^j(k+N-1) & \mathbf{U}_{o2}{}^j(k+N-1)^{\mathrm{T}}\end{bmatrix}^{\mathrm{T}}$, until the $\mathbf{J}_{\mathrm{p}}{}^j(k)$ is less than the threshold or the number of repetition exceeds J_{upper}, the iteration is stopped and the optimal control input $\mathbf{U}_o{}^*(k)\cdots\mathbf{U}^*{}_o(k+N-1)$ for the time k are obtained. Then only $\mathbf{U}_o{}^*(k)$ is used as the controller output, which is referred to as multiple step prediction and one step control. This process is called rolling optimization.

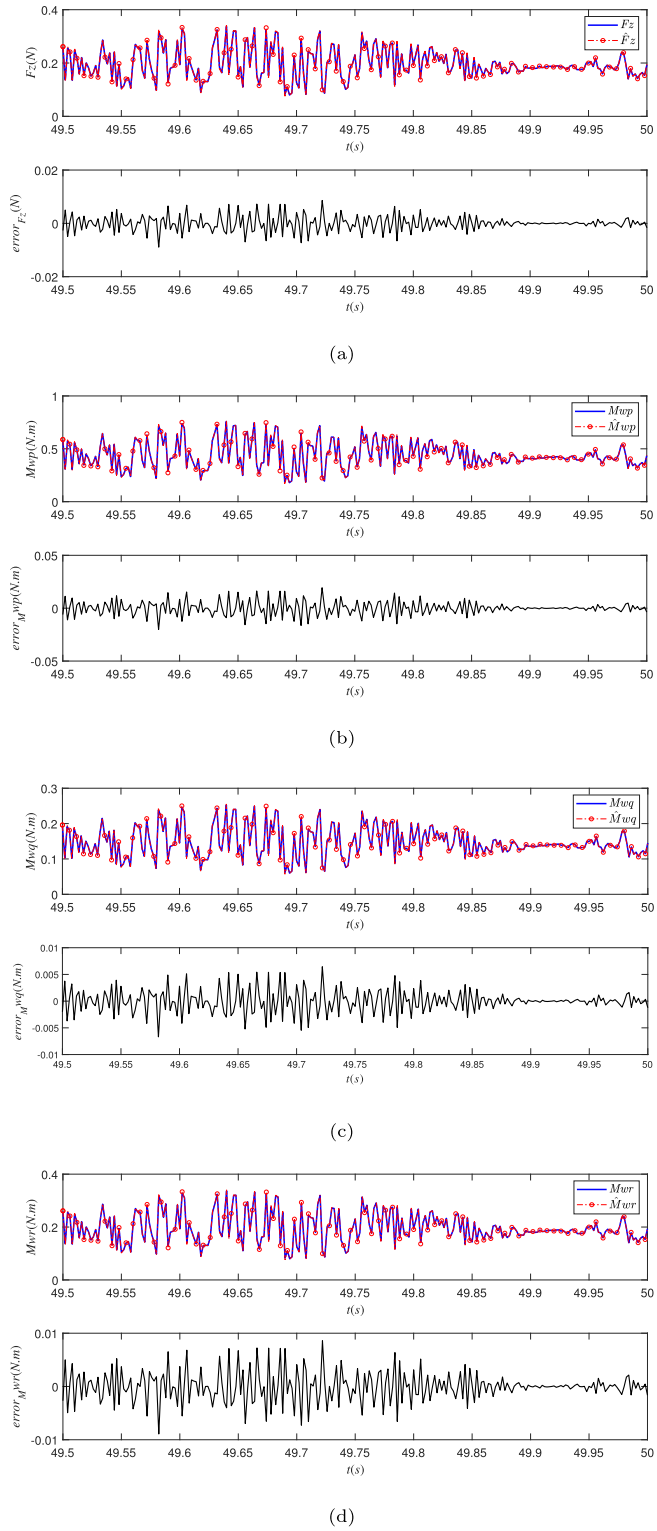


Fig. 5. The estimated results of the ESO, (a)F_z (b)M_{wp} (c)M_{wq} (d)M_{wr}.

4. Simulation results

4.1. Numerical simulation results

To show the superiority of the proposed AMPCESO controller, five baseline methods were selected based on their technical relevance to our innovation focus in addressing key challenges in QUAV control: (1) The Multi Objective Particle Swarm Optimization Proportional Integral Differential (MOPSOPID) controller [8] was included as it represents the classical control algorithms, providing a fundamental performance benchmark that highlights the advantages of MPC over conventional control strategies; (2) The IO tracking control [29] method was chosen as it exemplifies advanced disturbance estimation through interval analysis, offering a specialized comparison for evaluating ESO method in this paper; (3) The CMPCESO controller in [37] serves as a baseline for establishing the performance of conventional MPC combined with ESO in order to demonstrate the superiority of the proposed method; (4) AMPC approach in [26] exemplifies advanced data-driven model compensation techniques through neural network, providing a reference for evaluating control performance under both model inaccuracy and external disturbance; (5) FFTSESO method in [35] is concerned as a comparison to analysis of robustness against the proposed approach. This selection of five baseline methods covers the major representative approaches from traditional control optimization and specialized disturbance handling to advanced MPC implementations, providing comprehensive benchmark references.

Table 2
The MSE of tracking error for six methods in the presence of model inaccuracy and external disturbance.

Method	MOPSOPID	IO	CMPCESO	AMPC	FFTSESO	AMPCESO
MSE-vx	0.1119	0.1276	0.1423	0.1135	0.1198	**0.1116**
MSE-vy	0.1055	0.1191	0.1127	0.1031	0.1200	**0.1059**
MSE-vz	0.0129	0.0032	0.0013	9.0888e-05	0.006	**7.934e-05**
MSE-p	0.1324	0.0003	0.0003	0.0840	1.4491e-04	**1.6227e-04**
MSE-q	0.1225	0.0002	0.0008	0.0859	0.0604	**2.4459e-04**
MSE-r	0.0027	0.0003	4.6445e-05	2.3788e-04	2.6662e-6	**3.7141e-12**
MSE-x	0.3812	0.1615	0.2040	0.2012	0.1681	**0.1333**
MSE-y	0.1526	0.1562	0.1308	0.1352	0.1689	**0.1271**
MSE-z	0.0074	0.0039	0.0014	5.5937e-04	0.0067	**5.5704e-04**
MSE-dis	0.5412	0.3216	0.3362	0.3370	0.3101	**0.2609**

In the simulation, the parameters of the QUAV are given as $m = 0.886kg, g = 9.8m/s^2$, $J_{xx} = 0.016$, $J_{yy} = 0.016$, $J_{zz} = 0.0274$, $\Delta T = 0.004s$, $\rho_w = 1.29kg/m^3$, $S_w = 0.08$, $d_q = 0.4$. The wind parameters are given as $\varphi_{a,i}(i = 1,2,3) = 30°$, $\varphi_{b,i}(i = 1,2,3) = 30°$, $V_{\max 1} = 0.8$, $V_{\max 2} = 2$, $z_0 = 0.15$, $W_{6.096} = 5$, $\omega_n = \frac{\pi}{6}$. The expected speed is considered as $\mathbf{p}_T = [\pi\cos(2\theta)\cos(\theta) - \frac{\pi}{2}\sin(2\theta)\sin(\theta), \pi\sin(2\theta), 0.4\sin(t)]^T$ where $\theta = \frac{\pi t}{10}$.

The numerical simulation results of six methods under the same wind disturbance are shown in Figs. 5 and 6. The estimated results of the ESO are provided in Fig. 5, where the blue solid lines represent external wind disturbance, the red dash-dotted lines with circle marks represent the estimated values of the ESO, and the black solid lines represent the errors between the actual disturbance and the estimated disturbance of the ESO. The mean square error (MSE) of each error is 3.0439e-04(N), 3.3821e-05(N), 9.6203e-06(N), 4.8703e-05$(N \cdot m)$, 5.4114e-06$(N \cdot m)$, 9.6203e-06$(N \cdot m)$. We learn that the proposed ESO estimates the disturbance with nice performance. In Fig. 6, the black dash-dotted line represents the target trajectory, the blue dash-dotted line with circle marks represents the trajectory tracking result based on IO, the purple solid line represents the trajectory tracking result of MOPSOPID, the green solid line with star marks represents the trajectory tracking result of AMPC method, the light blue dashed line represents the trajectory tracking result of CMPCESO method, the red solid line with triangle marks represents the trajectory tracking result of FFTSESO method, and the pink dashed line with dots marks represents the trajectory tracking result proposed in this paper. From Table 2, it can be seen that the AMPCESO method has reduces the MSE-dis by 51.79 %, 18.86 %, 22.39 %, 22.58 %, and 15.86 % compared to MOPSOPID, IO, CMPCESO, AMPC and FFTSESO, respectively. Fig. 7 shows the control input of the method proposed in this paper. It can be seen that the actual control input is bounded and convergent. In conclusion, the AMPCESO method

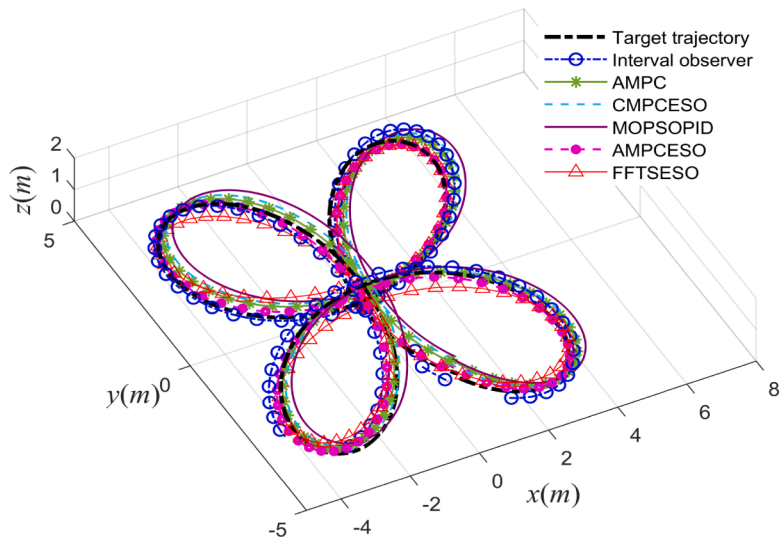


Fig. 6. The results of the trajectory tracking.

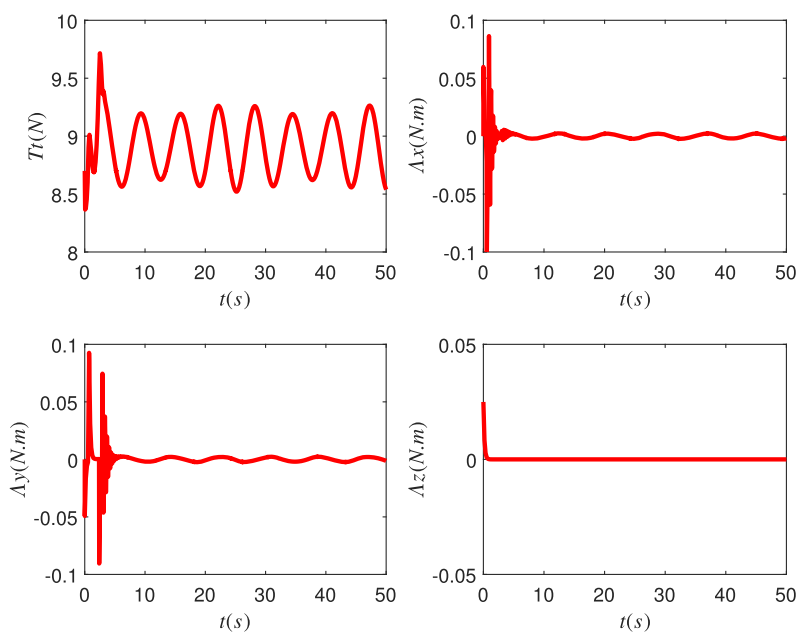


Fig. 7. Control input curves of AMPCESO.

Table 3
Parameters of the QUAV.

Parameter	Quality	Body radius	X-axis moment of inertia	Y-axis moment of inertia	Z-axis moment of inertia
value	0.886kg	0.225m	0.016	0.016	0.0274
Parameter	Gravitational acceleration 9.8055 m/s^2	Motor torque coefficient 1.982e-07 $kg.m/s^2$	Motor thrust coefficient 1.239e-05 $kg.m/s^2$	Motor throttle speed curve slope 718.078 rad/s	Motor speed throttle curve constant term 88.448 rad/s

Table 4
MSE of QUAV under six controllers.

Methods	MOPSOPID	IO	CMPCESO	AMPC	FFTSESO	AMPCESO
MSE(m)	0.3130	0.2694	0.2998	0.2916	0.2217	**0.1838**

significantly improves tracking accuracy in the QUAV flight with disturbance.

4.2. HIL experiment results

HIL Experiment was conducted on a QUAV using the PX4 open-source flight control platform, with a sampling frequency of 250 Hz and added wind disturbance. Using MATLAB/Simulink development environment, download it to the Pixhawk autopilot, and then install the Pixhawk autopilot on the hardware system of the Prometheus200 drone. Within the HIL environment, a desired trajectory denoted as Ref in Fig. 8 was programmed. This trajectory was set to maintain a constant altitude

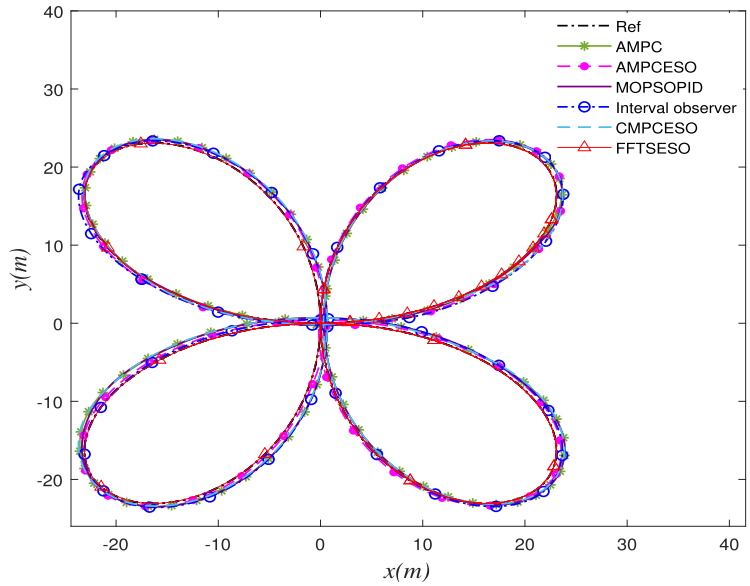


Fig. 8. The expected and actual trajectory tracking results of QUAV in the X-Y plane.

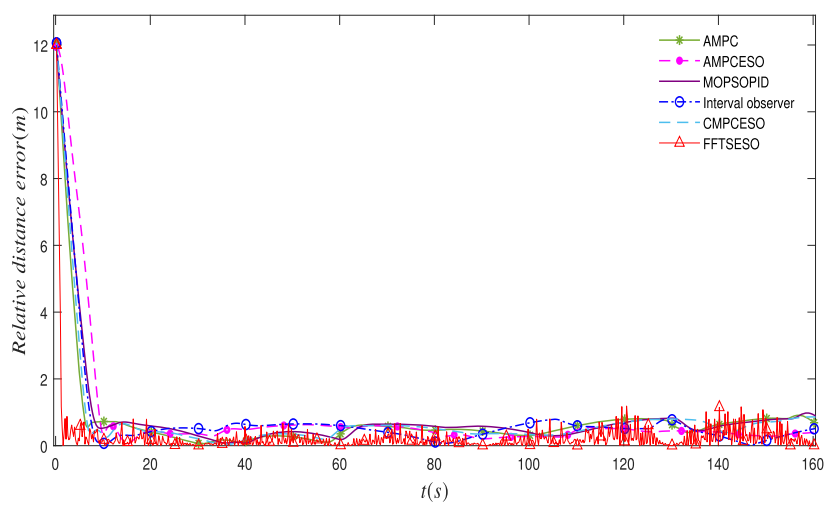


Fig. 9. The results of relative distance error.

of 5 m. The trajectory information was then transmitted to the QUAV via the QGroundControl (QGC) software platform for execution. Additionally, the QUAV parameters were configured according to Table 3.

Record the QUAV flight logs under the same wind disturbance environment in HIL to analyze the trajectory tracking performance of six controllers. Table 4 shows the MSE values for six controllers. Fig. 8 shows the expected and actual trajectory tracking results of QUAV in the X-Y plane. Fig. 9 shows the relative distance error of QUAV under the six controllers. The results show that all six controllers can achieve the desired trajectory under wind disturbance, but the error curve of the MOPSOPID controller fluctuates greatly. Although the FFTSESO method achieves a relatively small tracking MSE, it suffers from trajectory oscillations under wind disturbances. The CMPCESO controller relies on the system model and the tracking performance of the AMPC controller is slightly better than that of the CMPCESO, but the error curve of the AMPCESO controller is smoother, and the MSE is the smallest. Its tracking accuracy and robustness are significantly better than the other five controllers. Therefore, according to the HIL results, the AMPCESO controller outperforms MOPSOPID, CMPCESO, AMPC, IO and FFTSESO algorithms in terms of tracking accuracy and anti-disturbance ability.

Remark 1. It should be pointed out that the AMPCESO controller occupies 3.4 ms, which is affordable for 4ms sampling period on the GD32F470-based HIL testbench (240MHz ARM Cortex-M4).

5. Conclusion

The methodology of this paper addresses the trajectory tracking problem for the QUAV, which particularly treats both the time-varying model and external disturbance. In the proposed method, the neural network predictive model is updated in an on-line manner to accurately predict the QUAV output. The internal inaccuracy and the external disturbance are estimated by the ESO. Eventually, the AMPCESO is proposed to guarantee the tracking performance of the QUAV with disturbance. The additive composite Lyapunov stability theory is proved for the convergence of the tracking error. The results, including HIL Experiment, demonstrate the effectiveness and the superiority of the proposed method. Future research will focus on exploring multi-agent cooperative control for UAV swarm under this framework, enhancing the performance of autonomous trajectory planning and tracking in the unfamiliar environment.

CRediT authorship contribution statement

Kun Tian: Formal analysis; **Lin-Jie Hu:** Investigation; **Chao Bai:** Writing – review & editing; **Hai-Peng Ren:** Writing – review & editing, Supervision, Conceptualization.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This paper was supported in part by the National Natural Science Foundation of China [62203346], in part by the Xi'an University Science and Technology Talent Service Enterprise Project [24GXFW0031,24GXFW0027], in part by the Natural Science Foundation of Shaanxi Provincial Department of Science and Technology [2025JC-YBMS-661, 2025JC-YBMS-729, 2025JC-YBMS-483], in part by the Youth Innovation Team of Shaanxi Universities, and in part by National Key Laboratory of Land and Air based Information Perception and Control, China, No. AZ124004.

Appendix A.

The proof of the Theorem 1 is proved as follows.

Proof. Define the error as follows

$$\mathbf{e}(k+1) = \xi_L(k+1) - \hat{\xi}_L(k+1) \tag{A.1}$$

By substituting Eqs. (8) and (9) into Eq. (A.1), we have

$$\begin{aligned}\mathbf{e}(k+1) &= \mathbf{A}_0\xi_L(k) + \mathbf{B}_0\mathbf{U}_o(k) + \mathbf{\Delta}_h\mathbf{h}(k) + \mathbf{D}_0(k) - \mathbf{A}_0\hat{\xi}_L(k) - \mathbf{B}_0\mathbf{U}_o(k)\\ &\quad -\Delta_h\mathbf{h}(k) - \mathbf{D}_0(k) - \mathbf{L}_1(\mathbf{y}(k) - \hat{\mathbf{y}}(k))\\ &= \mathbf{A}_0\mathbf{e}(k) - \mathbf{L}_1(\mathbf{C}_0\xi_L(k) - \mathbf{C}_0\hat{\xi}_L(k))\\ &= (\mathbf{A}_0 - \mathbf{L}_1\mathbf{C}_0)\mathbf{e}(k)\end{aligned} \tag{A.2}$$

By sorting out Eq. (A.2), we get

$$\mathbf{e}(k+1) = (\mathbf{A}_0 - \mathbf{L}_1\mathbf{C}_0)\mathbf{e}(k) \tag{A.3}$$

Denote $\Theta = \mathbf{A}_0 - \mathbf{L}_1\mathbf{C}_0$, then Eq. (A.3) can be rewritten as

$$\mathbf{e}(\mathrm{k}+1) = \Theta\mathbf{e}(\mathrm{k}) \tag{A.4}$$

Define the Lyapunov function as

$$V_k = V[\mathbf{e}(k)] = \mathbf{e}^T(k)\mathbf{E}\mathbf{e}(k) \tag{A.5}$$

where $\mathbf{E}$ is the identity matrix

Then V_k is positive definite.

$$\begin{aligned}\Delta V_k &= V_{k+1} - V_k\\ &= \mathbf{e}^T(k+1)\mathbf{E}\mathbf{e}(k+1) - \mathbf{e}^T(k)\mathbf{E}\mathbf{e}(k)\\ &= (\Theta\mathbf{e}(k))^T\mathbf{E}(\Theta\mathbf{e}(k)) - \mathbf{e}^T(k)\mathbf{E}\mathbf{e}(k)\\ &= \mathbf{e}^T(k)[\Theta^T\mathbf{E}\Theta - \mathbf{E}]\mathbf{e}(k)\\ &= \mathbf{e}^T(k)[\Theta^T\Theta - \mathbf{E}]\mathbf{e}(k)\end{aligned} \tag{A.6}$$

It can be seen from Eq. (A.6) that if all the eigenvalues of Θ are negative, then $\Theta^{\mathrm{T}}\Theta - E$ is negative definite. Therefore, if the gain matrix $\mathbf{L}_1$ satisfies the condition $a > 2, b > 2, c > 2, d > 2, e > 2, f > 2, (a-1) < g \le \frac{a^2}{4}$, $(b-1) < h \le \frac{b^2}{4}$, $(c-1) < i \le \frac{c^2}{4}$ $(d-1) < j \le \frac{d^2}{4}$, $(e-1) < k \le \frac{e^2}{4}$, $(f-1) < l \le \frac{f^2}{4}$, then the corresponding $\Delta V_k < 0$, and Eq. (9) guarantees the estimated variables approach the $\hat{\mathbf{d}}(\mathrm{k})$ in a finite time.

□

Appendix B.

The proof of the Theorem 2 is proved as follows.

Proof. Define the additive composite Lyapunov function as

$$\begin{aligned}V_{total} &= V_{c_1} + V_{c_2}\\ V_{c_1} &= \tfrac{1}{2}\mathbf{E}_k{}^T(k)\mathbf{E}_k(k)\\ V_{c_2} &= \tfrac{1}{2}e^T(k)e(k)\end{aligned} \tag{B.1}$$

where $\mathbf{E}_k(k) = \hat{\mathbf{y}}_d(k)$, $\hat{\mathbf{y}}_d(\mathrm{k}) = \begin{bmatrix}\mathbf{Y}_o(k+1|k) + \hat{\mathbf{d}}_p(k+1|k) - \mathbf{p}_p(k+1|k)\\ \mathbf{Y}_o(k+2|k) + \hat{\mathbf{d}}_p(k+2|k) - \mathbf{p}_p(k+2|k)\\ \vdots\\ \mathbf{Y}_o(k+N|k) + \hat{\mathbf{d}}_p(k+N|k) - \mathbf{p}_p(k+N|k)\end{bmatrix}$.

By differentiating Eq. (B.1), we have

$$\dot{V}_{c_1} = \mathbf{E}_k{}^T(k)\dot{\mathbf{E}}_k(k) \tag{B.2}$$

Substituting Eq. (14) into $\mathbf{E}_k(k)$, we have

$$\dot{\mathbf{E}}_k(k) = \frac{\partial\mathbf{E}_k(k)}{\partial k} = \frac{\partial\mathbf{E}_k(k)}{\partial\hat{\mathbf{y}}_d(k)}\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\frac{\partial\mathbf{U}_o(k)}{\partial k} = \frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\Delta\mathbf{U}_o(k) \tag{B.3}$$

where

$$\Delta\mathbf{U}_o(k) = -lr_{uc}\frac{\partial\mathbf{J}}{\partial\mathbf{U}_o(k)} \tag{B.4}$$

To simplify calculation, the cost function of Eq. (13) can be written as

$$\mathbf{J} = \hat{\mathbf{y}}_d^{\mathrm{T}}(k)\hat{\mathbf{y}}_d(k) \tag{B.5}$$

According to Eq. (B.5), we have

$$\begin{aligned}\frac{\partial\mathbf{J}}{\partial\mathbf{U}_o(k)} &= \left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{U}_o(k)}\right]^T\hat{\mathbf{y}}_d(k) + \hat{\mathbf{y}}_d^{\mathrm{T}}(k)\left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{U}_o(k)}\right]\\ &= \left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\right]^T\hat{\mathbf{y}}_d(k) + \hat{\mathbf{y}}_d^{\mathrm{T}}(k)\left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\right]\end{aligned} \tag{B.6}$$

Substituting Eqs. (B.3), (B.4), (B.6) into Eq. (B.2), we get

$$\dot{V}_{c_1} = -lr_{uc}\hat{\mathbf{y}}_d^T(k)\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\left\{\left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\right]^T\hat{\mathbf{y}}_d(k) + \hat{\mathbf{y}}_d^T(k)\left[\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)}\right]\right\} \tag{B.7}$$

Denote $\hat{\mathbf{y}}_d^T(\mathrm{k})\frac{\partial\hat{\mathbf{y}}_d(k)}{\partial\mathbf{Y}_o(k)}\frac{\partial\mathbf{Y}_o(k)}{\partial\mathbf{U}_o(k)} = \Phi(k)$, we have

$$\begin{aligned}\dot{V}_{c_1} &= -lr_{uc}\Phi(k)\left(\Phi(k)^{\mathrm{T}} + \Phi(k)\right)\\ &= -lr_{uc}\left(1 + \Phi^2(k)\right)\end{aligned} \tag{B.8}$$

According to Theorem 1, we have

$$\dot{V}_{c_2} \le \left(\Theta^{\mathrm{T}}\Theta - E\right)\|e(k)\|^2 \tag{B.9}$$

In summary, we have

$$\dot{V}_{total} = -lr_{uc}(1 + \Phi^2(k)) + \left(\Theta^{\mathrm{T}}\Theta - E\right)\|e(k)\|^2 \tag{B.10}$$

Therefore, according to Eq. (B.10), as long as $lr_{uc} > 0$ and $\Theta^{\mathrm{T}}\Theta - E < 0$ are satisfied, the AMPCESO guarantees that the tracking error and observation error are asymptotically convergent to zero.

□

The glossary of variables is provided in Table B.5.

Table B.5
Glossary of variables.

$\Upsilon = [x, y, z]^T$	The position vectors of UAV	$\mathbf{d}_F = [F_X, F_Y, F_Z]^T$	The wind disturbance forces
$\mathbf{v} = [u, v, w]^T$	The velocity vectors of UAV	$\mathbf{d}_M = [M_{wp}, M_{wq}, M_{wr}]^T$	The aerodynamic torque
$\Theta = [\phi, \theta, \varphi]^T$	Attitude angle vector	$\xi_L, \mathbf{X}_L = [u, v, w, p, q, r]^T$	System state variables
$\sigma = [p, q, r]^T$	Attitude angular rate vector	$\Lambda = [\Lambda_x, \Lambda_y, \Lambda_z]^T$	Torque acting on the QUAV
m	The mass of the QUAV	g	Gravity accelerant
$J = diag\{J_{xx}, J_{yy}, J_{zz}\}$	Inertia matrix	R_b^g	The rotation matrix
B	Attitude kinematic matrix	T_t	Pulling force
ΔT	Sampling time	$\mathbf{L}_1$	The gain matrix of the ESO
$\hat{\xi}_L$	The estimation of ξ_L by ESO	$\hat{y}$	The output estimation by ESO
$\mathbf{P}_1, \mathbf{P}_2$	The inputs of the neural network	$\mathbf{Y}_{o1}, \mathbf{Y}_{o2}$	The outputs of the neural network
$\mathbf{IW}_1, \mathbf{IW}_2, \mathbf{LW}_1, \mathbf{LW}_2$	The weight matrixes of the neural network	$\mathbf{b}_{11}, \mathbf{b}_{21}, \mathbf{b}_{12}, \mathbf{b}_{22}$	the bias vectors of the neural network
$\mathbf{U}_O$	The nominal control inputs	$\mathbf{U}_O^*$	The MPC controller output
$\mathbf{A}, \mathbf{A}_0, \mathbf{B}_0, \mathbf{B}_d, \mathbf{C}, \mathbf{d}, \mathbf{D}, \mathbf{D}_0, \mathbf{h}, \Delta_h$	Intermediate variable		

References

[1] W. Park, X. Wu, D. Lee, et al., Modeling and control of a top-loading fully-actuated cargo transportation multirotor, IEEE Rob. Autom. Lett. 8 (2023) 5807–5814.
[2] D. Rohr, M. Studiger, T. Stastny, et al., Nonlinear model predictive velocity control of a VTOL tiltwing UAV, IEEE Rob. Autom. Lett. 6 (2021) 5776–5783.
[3] T. Tomic, K. Schmid, L. P, et al., Toward a fully autonomous UAV: research platform for indoor and outdoor urban search and rescue, IEEE Rob. Autom. Mag. 19 (2012) 46–56.
[4] H. Oh, S. Kim, H. Shin, et al., Coordinated standoff tracking of moving target groups using multiple UAVs, IEEE Trans. Aerosp Electron. Syst. 51 (2015) 1501–1514.
[5] S. Sun, Y. Liu, S. Guo, et al., Observation-driven multiple UAV coordinated standoff target tracking based on model predictive control, Tsinghua Sci. Technol. 27 (2022) 948–963.
[6] Z. Liu, L. Xiang, Z. Zhu, Cooperative standoff target tracking using multiple fixed-Wing UAVs with input constraints in unknown wind, Drones 7 (2023) 593.
[7] A.S. Elkhatem, S.N. Engin, Robust LQR and LQR-PI control strategies based on adaptive weighting matrix selection for a UAV position and attitude tracking control, Alexandria Eng. J. 61 (2022) 6275–6292.
[8] N.H. Sahrir, M.A.M. Basri, PSO–PID Controller for quadcopter UAV: index performance comparison, Arabian J. Sci. Eng. 48 (2023) 15241–15255.
[9] S. Mobayen, F.F.M. El-Sousy, K.A. Alattas, et al., Adaptive fast-reaching nonsingular terminal sliding mode tracking control for quadrotor UAVs subject to model uncertainties and external disturbances, Ain Shams Eng. J. 14 (2023) 102059.
[10] C. Bao, Y. Guo, L. Luo, et al., Design of a fixed-wing UAV controller based on adaptive backstepping sliding mode control method, IEEE Access 9 (2021) 157825–157841.
[11] X. Zhang, Y. Zhuang, X. Zhang, et al., A novel asymptotic robust tracking control strategy for rotorcraft UAVs, IEEE Trans. Autom. Sci. Eng. 20 (2023) 2338–2349.
[12] L. Cao, B. Xiao, M. Golestani, et al., Faster fixed-time control of flexible spacecraft attitude stabilization, IEEE Trans. Ind. Inf. 16 (2020) 1281–1290.
[13] B. Xiao, X.W. Wu, L. Cao, et al., Prescribed time attitude tracking control of spacecraft with arbitrary disturbance, IEEE Trans. Aerosp Electron. Syst. 58 (2022) 2531–2540.
[14] Y. Kang, J.K. Hedrick, Linear tracking for a fixed-wing UAV using nonlinear model predictive control, IEEE Trans. Control Syst. Technol. 17 (2009) 1202–1210.
[15] U. Eren, A. Prach, B.B. Koçer, et al., Model predictive control in aerospace systems: current state and opportunities, J. Guid. Contr. Dyn. 40 (2017) 1541–1566.
[16] B. Lindqvist, S.S. Mansouri, A. Agha-mohammadi, et al., Nonlinear MPC for collision avoidance and control of UAVs with dynamic obstacles, IEEE Rob. Autom. Lett. 5 (2020) 6001–6008.
[17] H. Ülker, C. Baykara, C. Özsoy, Design of MPCs for a fixed wing UAV, Aircraft Eng. Aerospace Technol. 89 (2017) 893–901.
[18] A. Eskandarpour, I. Sharf, A constrained error-based MPC for path following of quadrotor with stability analysis, Nonlinear Dyn. 99 (2020) 899–918.
[19] M. Aliyari, W.K. Wong, Y. Bouteraa, et al., Design and implementation of a constrained model predictive control approach for unmanned aerial vehicles, IEEE Access 10 (2022) 91750–91762.
[20] J. Muliadi, B. Kusumoputro, Neural network control system of UAV altitude dynamics and its comparison with the PID control system, Journal of Advanced Transportation 2018 (2018) 1–18.
[21] T. Dierks, S. Jagannathan, Output feedback control of a quadrotor UAV using neural networks, IEEE Trans. Neural Netw. 21 (2009) 50–66.
[22] H. Razmi, S. Afshinfar, Neural network-based adaptive sliding mode control design for position and attitude control of a quadrotor UAV, Aerosp. Sci. Technol. 91 (2019) 12–27.
[23] W. Gu, K.P. Valavanis, M.J. Rutherford, et al., UAV model-based flight control with artificial neural networks: a survey, J. Intell. Rob. Syst. 100 (2020) 1469–1491.
[24] B. Zhang, X. Sun, S. Liu, et al., Recurrent neural network-based model predictive control for multiple unmanned quadrotor formation flight, Int. J. Aerospace Eng. 2019 (2019) 1–18.
[25] S. Bansal, A.K. Akametalu, F.J. Jiang, et al., Learning quadrotor dynamics using neural network for flight control, IEEE 55th Conference on Decision and Control (CDC) - (2016) 4653–4660.
[26] B. Jiang, B. Li, W. Zhou, et al., Neural network based model predictive control for a quadrotor UAV, Aerospace 9 (2022) 460.
[27] Y.C. Lin, D.D. Chen, M.S. Chen, et al., A precise BP neural network-based online model predictive control strategy for die forging hydraulic press machine, Neural Comput. Appl. 29 (2018) 585–596.
[28] G.E.A. Mustafa, Z.A. Ali, S.A. Zulkifli, et al., Performance evaluation of different control methods for an underactuated quadrotor unmanned aerial vehicle (QUAV) with position estimator and disturbance observer, Math. Prob. Eng. 50 (2021) 8791620.1–22.
[29] K. Yan, J.R. Zhang, H.P. Ren, Interval observer-based robust trajectory tracking control for quadrotor unmanned aerial vehicle, Int. J. Contr. Autom. Syst. 22 (2024) 288–300.
[30] B. Li, W. Gong, Y. Yang, et al., Appointed fixed time observer-based sliding mode control for a quadrotor UAV under external disturbances, IEEE Trans. Aerosp. Electron. Syst. 58 (2022) 290–303.
[31] R. Ortega, E. Nuño, A. Bobtsov, Distributed observers for LTI systems with finite convergence time: a parameter estimation-based approach, IEEE Trans. Automat. Contr. 66 (2021) 4067–4974.
[32] A. Pyrkin, A. Bobtsov, R. Ortega, et al., An adaptive observer for uncertain linear time-varying systems with unknown additive perturbations, Automatica 147 (2023) 110677.
[33] W. Zhao, Z. Meng, K. Wang, et al., Backstepping control of an unmanned helicopter subjected to external disturbance and model uncertainty, Appl. Sci. 11 (2021) 5331.
[34] S. Zhao, Y. Xia, L. Ma, et al., Finite-time height control of quadrotor UAVs, Appl. Sci. 13 (2023) 7914.
[35] N.S. Wang, R. Hamrah, A.K. Sanyal, et al., Geometric extended state observer on TSE(3) with fast finite-time stability: theory and validation on a rotorcraft aerial vehicle, Aerosp. Sci. Technol. 155 (2024) 109596.
[36] S.A. Emami, A. Rezaeizadeh, Adaptive model predictive control-based attitude and trajectory tracking of a VTOL aircraft, IET Contr. Theory Appl. 12 (2018) 2031–2042.
[37] B. Zhang, X. Sun, S. Liu, et al., Adaptive model predictive control with extended state observer for multi-UAV formation flight, Int. J. Adapt. Contr. Signal Process. 34 (2020) 1341–1358.
[38] J. Shen, B. Wang, B.M. Chen, et al., Review on wind resistance for quadrotor UAVs: modeling and controller design, Unmanned Syst. 11 (2023) 5–15.
[39] J. Wang, J. Yang, Z. Yang, Dynamics modeling and simulation of multi-rotor UAV based on the composite wind field model, 13th International Conference on Reliability, Maintainability, and Safety (ICRMS) - (2022) 127–134.
[40] W. Ding, D. Wei, Y. Diao, et al., Research on trajectory tracking control of ocean unmanned aerial vehicles based on disturbance observer and nonlinear sliding mode, Ocean Eng. 293 (2024) 116682.